

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Comparative Analysis

COURSE NAME: Data Structures

COURSE CODE: CSA0302

COURSE OUTCOME COVERED

CO: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data **(BL3)**

Assessment Tool Weightage: 30%

QUESTIONS: EACH QUESTION CARRIES: 10 MARKS DURATION: 1 HOUR

32) Compare Single-core (sequential) sorting and multi-core sorting based on the following:

- a. Performance improvement
- b. Synchronization overhead
- c. Practical challenges

Comparison of Single-Core (Sequential) Sorting and Multi-Core Sorting:

Introduction

Sorting is the process of arranging data elements in a particular order, usually ascending or descending order.

Examples:

45, 12, 78, 23, 9

Sorting:

9, 12, 23, 45, 78

In a single-core/sequential system, sorting is performed by one processor/core, so the operations are executed one after another.

In a multi-core system, the sorting problem can be divided into smaller parts and multiple CPU cores can process those parts simultaneously (in parallel).

Single-Core / Sequential Sorting

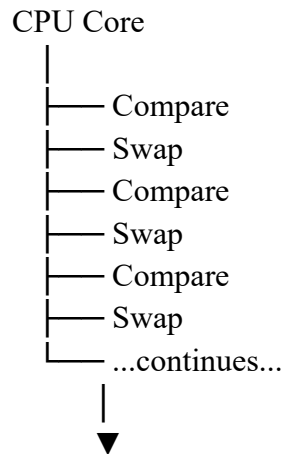
In sequential sorting, **one CPU core** performs all sorting operations sequentially.

Example

Suppose we have:

[8, 3, 7, 2, 6, 1, 5, 4]

A single core handles the entire array:



Sorted Array

Only **one stream of execution** is active.

Multi-Core Sorting

Multi-core sorting divides the input data into multiple sections and assigns different sections to different CPU cores.

For example, with **4 CPU cores**:

Thus, several sorting tasks can be executed **at the same time**.

Comparison at a Glance:

Feature	Single-Core Sorting	Multi-Core Sorting
Number of cores	1	2, 4, 8, 16, etc.
Execution	Sequential	Parallel
Performance	Lower for large data	Higher for large data
Parallelism	No	Yes
Synchronization	Very low	Higher
Programming complexity	Simple	More complex
Memory sharing	Simple	Requires careful management
Scalability	Limited	Better
Suitable for	Small/medium data	Large datasets
Overhead	Low	Thread, communication & synchronization overhead
Implementation	Easier	Difficult
Hardware requirement	Single-core CPU	Multi-core CPU

Performance is one of the most important differences between sequential and multi-core sorting.

If the input size becomes very large, the execution time increases.

For example:

Small Data → Fast

Medium Data → Moderate

Large Data → Slow

Very Large → Very Slow

Multi-Core Performance

In multi-core sorting, the input is divided.

Instead of one core doing all the work, several cores work simultaneously.

Theoretical Performance

If a task can be perfectly divided among **P cores**, the ideal execution time can approach:

For example:

Single Core:

100 seconds

4 Cores:

Ideal $\approx 100 / 4$

≈ 25 seconds

Important: In real systems, it will usually take **more than 25 seconds** because of synchronization, communication, memory access, load imbalance, and merging.

Amdahl's Law

The maximum speedup of a parallel program can be explained using **Amdahl's Law**:

Where:

- **P** = fraction of the program that can be parallelized
- **N** = number of processor cores

Example

Suppose 90% of sorting work can be parallelized and we use 4 cores:

So the speedup is approximately **3.08×**, not 4×

Because some operations cannot be performed in parallel.

Synchronization Overhead

Single-Core

In a single-core system, synchronization is **very low** because there is normally only one execution thread modifying the sorting data.

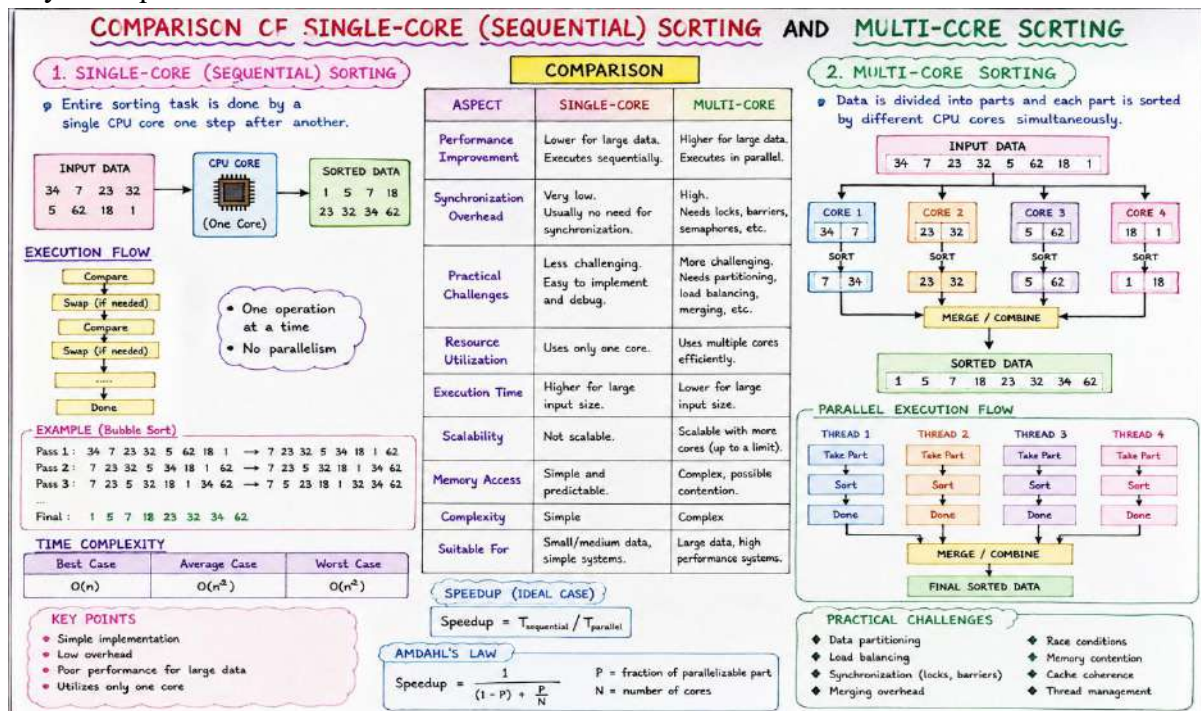
There is no need for multiple cores to coordinate their work.

Advantages

- Less synchronization
- No thread coordination
- No race conditions between sorting threads
- Simpler implementation
- Easier debugging

Multi-Core Synchronization

In multi-core sorting, several threads may access shared resource. Therefore, synchronization may be required.



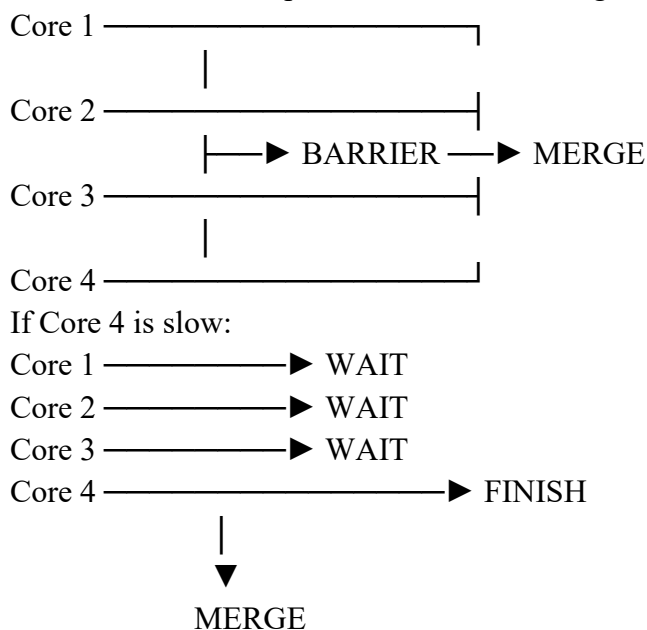
Common synchronization mechanisms

- ◇ Mutex Locks
- ◇ Semaphores
- ◇ Thread joining

Barrier Synchronization

A common example in parallel sorting is a **barrier**.

All threads must complete their current sorting work before the next phase starts.



This is called **synchronization overhead**.

Multi-core sorting provides better performance, but implementing it is more difficult.

The input data must be divided among the cores.

[1 2 3 4 5 6 7 8]

Core 1 $\rightarrow$ [1 2]

Core 2 $\rightarrow$ [3 4]

Core 3 $\rightarrow$ [5 6]

Core 4 $\rightarrow$ [7 8]

If the data is not divided properly, some cores may have more work.

Suppose:

Core 1 → 

[illegible]Core 3 →

--	--	--	--

Core 4 →

Core 1 has much more work.

Other cores may finish early and wait.

Ideal situation:

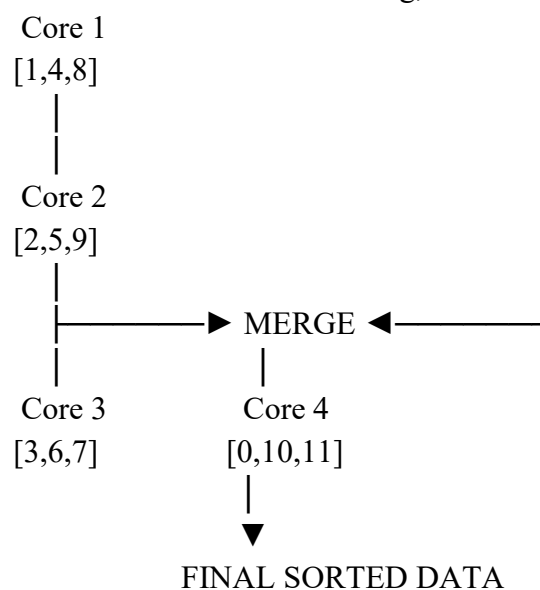
Core 1 →

Core 2 →

[illegible][illegible]

Therefore, **load balancing** is very important.

After individual cores finish sorting, their results must be combined.

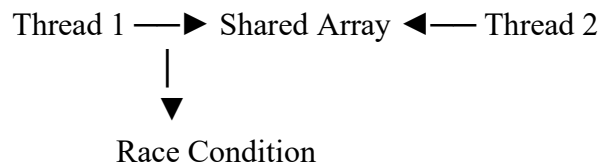


The merge operation takes additional time.

4. Race Conditions

If multiple threads access shared data incorrectly, a **race condition** can occur.

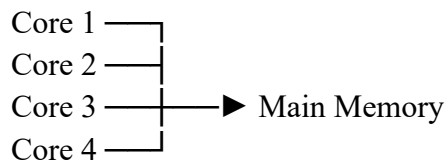
Example:



The final result may become incorrect if synchronization is not properly implemented.

5. Memory Access

Multiple cores may access the same memory.

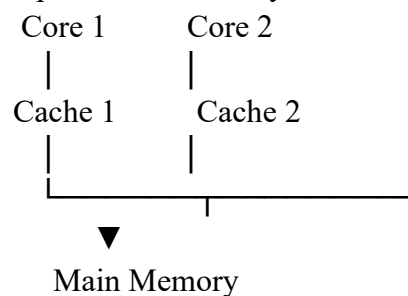


This can create **memory bandwidth contention**.

Even though there are many cores, the memory system may become a bottleneck.

6. Cache Coherence

Each processor core may have its own cache.



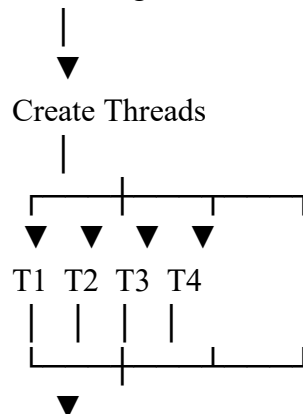
When multiple cores modify related data, the system must keep their cached copies consistent.

This creates additional overhead.

7. Thread Creation Overhead

Creating and managing threads takes time.

Main Program



Join Threads

For very small arrays, this overhead can be greater than the performance benefit.

Therefore:

Parallel sorting is not always faster than sequential sorting.

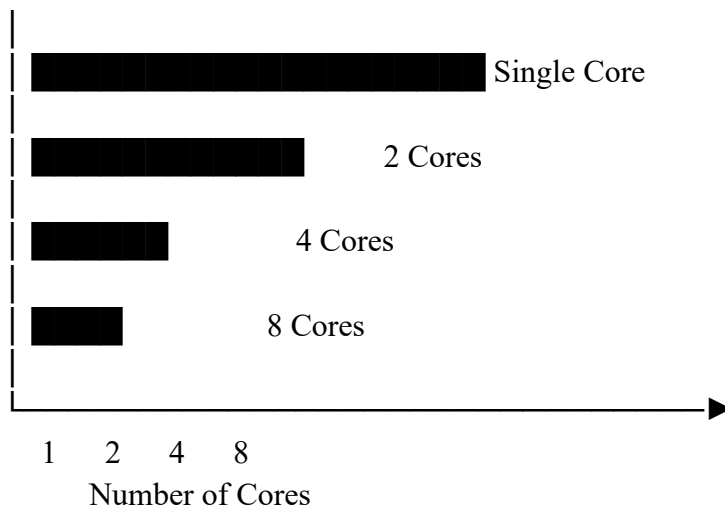
Detailed Comparison

Parameter	Single-Core / Sequential	Multi-Core / Parallel
Processing	One operation stream	Multiple operation streams
Cores	One	Multiple
Parallelism	None	High
Execution time	Higher for large datasets	Lower for suitable large datasets
Synchronization	Minimal	Required
Thread management	Not required	Required
Data partitioning	Not required	Required
Load balancing	Not applicable	Very important
Merge phase	Usually simple/not separate	Often required
Race conditions	Very low concern	Possible
Memory contention	Lower	Higher
Cache issues	Less complex	More complex
Programming	Easy	More difficult
Debugging	Easy	Difficult
Scalability	Limited	Better
Hardware	Simple	Multi-core CPU required
Small datasets	Often efficient	Parallel overhead may dominate
Large datasets	Can be slower	Usually better
Energy/performance	Depends on workload	Can improve throughput
Implementation cost	Low	Higher

Performance Graph Concept

For a sufficiently large dataset, increasing the number of cores can reduce execution time, although the improvement eventually becomes limited.

Execution Time



The graph is not perfectly linear because of:

- Synchronization
- Communication
- Memory bandwidth
- Load imbalance
- Serial portions
- Merge operations

Example: Merge Sort

- **Merge Sort** is particularly suitable for demonstrating parallel sorting.

Sequential Merge Sort

- Everything is performed by the same core.

Parallel Merge Sort

- This demonstrates how sorting work can be distributed across cores.

Advantages of Single-Core Sorting

Simple implementation

Easy debugging

Low synchronization overhead

Low thread-management overhead

Predictable execution

Good for small datasets

Requires less complex hardware

Disadvantages

Slower for very large datasets

Does not utilize multiple CPU cores

Limited scalability

Cannot exploit parallelism

Advantages of Multi-Core Sorting

Faster processing for large datasets

Utilizes multiple CPU cores

Better throughput

Can handle large datasets efficiently

Good scalability

Suitable for high-performance computing

Disadvantages

Synchronization overhead

Thread management required

More complex programming

Load balancing problems

Race conditions may occur

Memory contention

Merge overhead

Difficult debugging

Real-World Applications

Single-Core / Sequential Sorting

Useful when:

- Dataset is small
- Simplicity is important
- Parallel hardware is unavailable
- Parallel overhead is greater than the expected benefit

Multi-Core Sorting

Useful in:

- Database systems
- Big Data processing
- Search engines
- Large-scale simulations
- High-performance computing

Code of Conduct:

I B. Siva Parvathi(192525194) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Siva Parvathi

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand